7

PHOTOMOSAICS

When I was in the sixth grade, I saw a picture like the one shown in **Figure 7-1** but couldn't quite figure out what it was. After squinting at it for a while, I eventually figured it out. (Turn the book upside down, and view it from across the room. I won't tell anyone.)

The puzzle works because of how the human eye functions. The low-resolution, blocky image shown in the figure is hard to recognize up close, but when it is seen from a distance, you know what it represents because your eyes perceive less detail, which makes the edges smooth.

A *photomosaic* is an image that works according to a similar principle. You take a *target* image, split it into a grid of rectangles, and replace each rectangle with another, smaller image that matches that section of the target. When you look at a photomosaic from a distance, all you see is the target image, but if you come closer, the secret is revealed: the image actually consists of many tiny images!

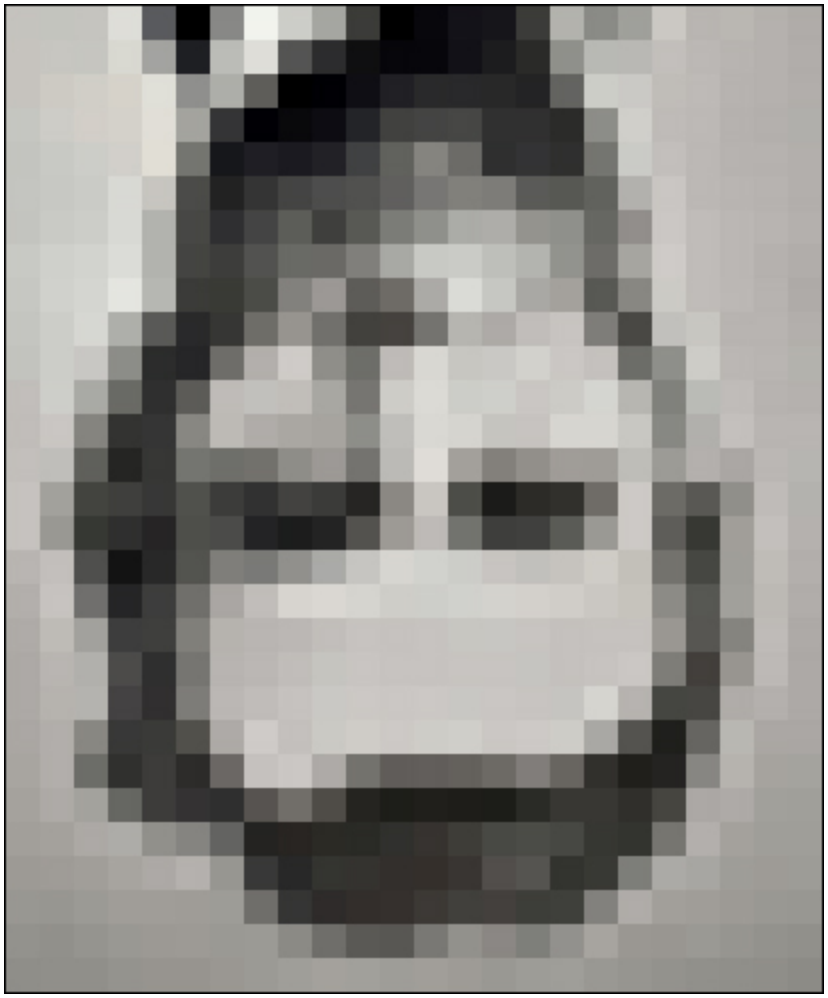

Figure 7-1: *A puzzling image*

In this project, you'll create a photomosaic using Python. You'll divide a target image into a grid and replace each block in the grid with a suitable image to create a photomosaic of the original. You'll be able to specify the grid dimensions and choose whether input images can be reused in the mosaic.

As you work on the project, you'll learn how to do the following:

- Create images using the Python Imaging Library (PIL).
- Compute the average RGB value of an image.
- Crop images.
- Replace part of an image by pasting in another image.
- Compare RGB values using a measurement of average distance in three dimensions.

- Use a data structure called a *k-d tree* to efficiently find the image that best matches a section of the target image.

How It Works

To create a photomosaic, begin with a blocky, low-resolution version of the target image (because the number of tile images would be too great in a high-resolution image). The user inputs the dimensions $M \times N$ (where M is the number of rows and N is the number of columns) of the mosaic. Next, build the mosaic according to this methodology:

1. Read the input images, which will be drawn on to replace the tiles in the original image.
2. Read the target image and split it into an $M \times N$ grid of tiles.
3. For each tile, find the best match from the input images.
4. Create the final mosaic by arranging the selected input images in an $M \times N$ grid.

Splitting the Target Image

We'll start by looking at how to split the target image into an $M \times N$ grid of tiles. Follow the scheme shown in **Figure 7-2**.

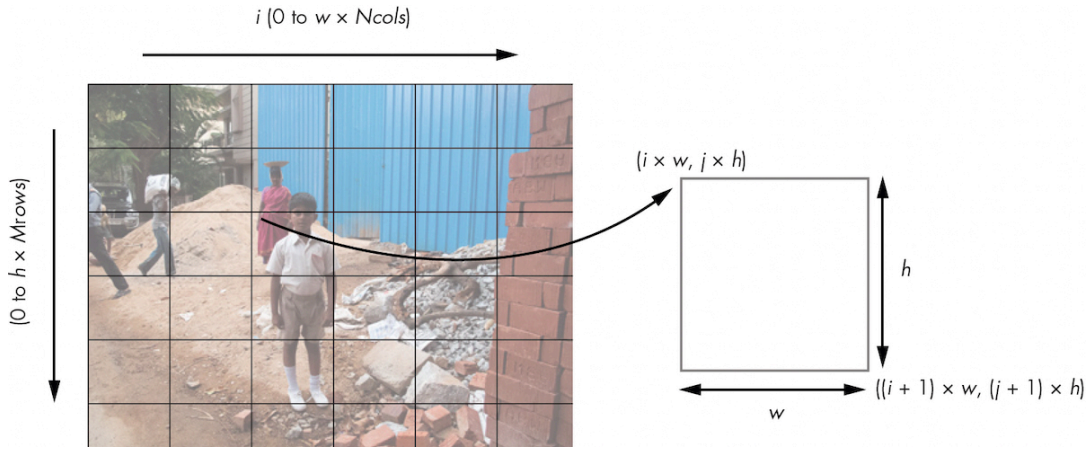

Figure 7-2: *Splitting the target image*

We split the original image into a grid of tiles with N columns arranged along the x-axis and M rows arranged along the y-axis. Each tile is represented by an index (i, j) and is w pixels wide and h pixels high. According to this scheme, the original image is $w \times N$ pixels wide and $h \times M$ pixels high.

The right side of **Figure 7-2** shows how to calculate the pixel coordinates for a single tile from this grid. The tile with index (i, j) has a top-left corner coordinate of $(i \times w, j \times h)$ and a bottom-right corner coordinate of $((i + 1) \times w, (j + 1) \times h)$. These coordinates can be used with the PIL to crop and create tiles from the original image.

Averaging Color Values

Every pixel in an image has a color that can be represented numerically by its red, green, and blue values. In this case, you are using 8-bit images, so each of these three color components has an 8-bit value in the range [0, 255]. You can therefore determine the average color of an image by taking the average of the red, green, and blue values for all of the image's pixels. Given an image with a total of N pixels, the average RGB is calculated as follows:

$$(r,g,b)_{\text{avg}} = \left(\frac{r_1 + r_2 + \ldots + r_N}{N}, \frac{g_1 + g_2 + \ldots + g_N}{N}, \frac{b_1 + b_2 + \ldots + b_N}{N} \right)$$

Like the RGB for an individual pixel, the average RGB for a whole image is a triplet, not a scalar or single number, because the averages are calculated separately for each color component. You calculate the average RGB to match the tiles from the target image with replacements from among the input images.

Matching Images

For each tile in the target image, you need to find a matching image from the images in the input folder specified by the user. To determine

whether two images match, use the average RGB values. The best match is the input image with the average RGB value closest to that of the tile from the target image.

The simplest way to find the best match is to calculate the distance between the average RGB values as if they were points in 3D space. After all, each average RGB consists of three numbers, which you can think of as x-, y-, and z-axis coordinates. You can thus use the following formula from the geometry for calculating the distance between two 3D points:

$$D_{1,2} = \sqrt{(r_1 - r_2)^2 + (g_1 - g_2)^2 + (b_1 - b_2)^2}$$

Here you compute the distance between the points (r_1, g_1, b_1) and (r_2, g_2, b_2). Given a target average RGB value (r_1, g_1, b_1), you can plug a list of average RGB values from the input images into the previous formula as (r_2, g_2, b_2) to find the closest matching image. However, there might be hundreds or even thousands of input images to check. We should therefore give some thought to how to efficiently search the set of input images to find the best match.

Using Linear Search

The simplest approach to searching for a match is a *linear search*. In this method, you just iterate through all the RGB values one by one and find the one with the minimum distance to the query value. The code will look something like this:

```
min_dist = MAX_VAL

for val in vals:

    dist = distance(query, val)

    if dist < MAX_VAL:
```

```
        min_dist = dist
```

You go through each value in the list `vals` one by one and calculate the distance between that value and `query`. If the result is less than `min_dist` (which was initialized as the maximum possible distance between two points), you update `min_dist` with the distance you just calculated. After checking every item in `vals`, `min_dist` will contain the smallest distance in the whole dataset.

Although a linear search method is easy to understand and implement, it isn't very efficient. If there are *N* values in the `vals` list, the search will take an amount of time proportional to *N*. You can achieve much better performance with a different data structure and search algorithm.

Using k-d Trees

A *k-d tree*, or *k-dimensional tree*, is a data structure that partitions a space of *k* dimensions—that is, it divides the space into a number of non-overlapping subspaces. This data structure provides a way to sort and search through datasets whose members are points in *k*-dimensional space. The dataset is represented as a *binary tree*: each point in the dataset becomes a node in the tree, and each node can have two child nodes. In other words, each node in the tree divides the space into two parts, called *subtrees*. One part points to the left of the node (the node's left child and its descendants), and the other points to the right of the node (the node's right child and its descendants).

Each node of the tree is associated with one of the dimensions of the space, and that's the dimension used to determine if points belong in the node's left subtree or right subtree. If a node is associated with the x-axis, for example, points whose x-values are less than that node's x-value will be put in the node's left subtree, and points whose x-values are greater than the node's x-value will be put in the right subtree. A common method to select the dimension associated with each node is

to cycle through them as you move down the levels of the tree. For example, in the case of a three-dimensional k-d tree, you could set the dimensions to be x, y, z, x, y, z, and so on, moving down the tree. Nodes at the same tree height will have the same splitting dimension.

Let's look at a simple example of a k-d tree. Say you have the following set of points, *P*:

P = {(5, 3), (2, 4), (1, 2), (6, 6), (7, 2), (4, 6), (2, 8)}

In this case, you'd build a two-dimensional k-d tree, since each member of *P* describes a point in two-dimensional space. You start by associating the first node, or the *root* node, (5, 3), with the x-dimension. Then you add the next point, (2, 4), as a left child of the root node, since the point's x-coordinate, 2, is less than 5, the x-coordinate of the root. The node (2, 4), being on the second level of the k-d tree, will use the y-dimension for partition. The next point in the list is (1, 2). Starting again at the root, 1 < 5, so you go to the left child of the root node. You then compare (1, 2) with (2, 4) using the y-dimension. Since 2 < 4, you add (1, 2) as the left child of (2, 4).

If you continue in this fashion for all the points in *P*, you'll create the tree and space partitioning shown in **Figure 7-3**.

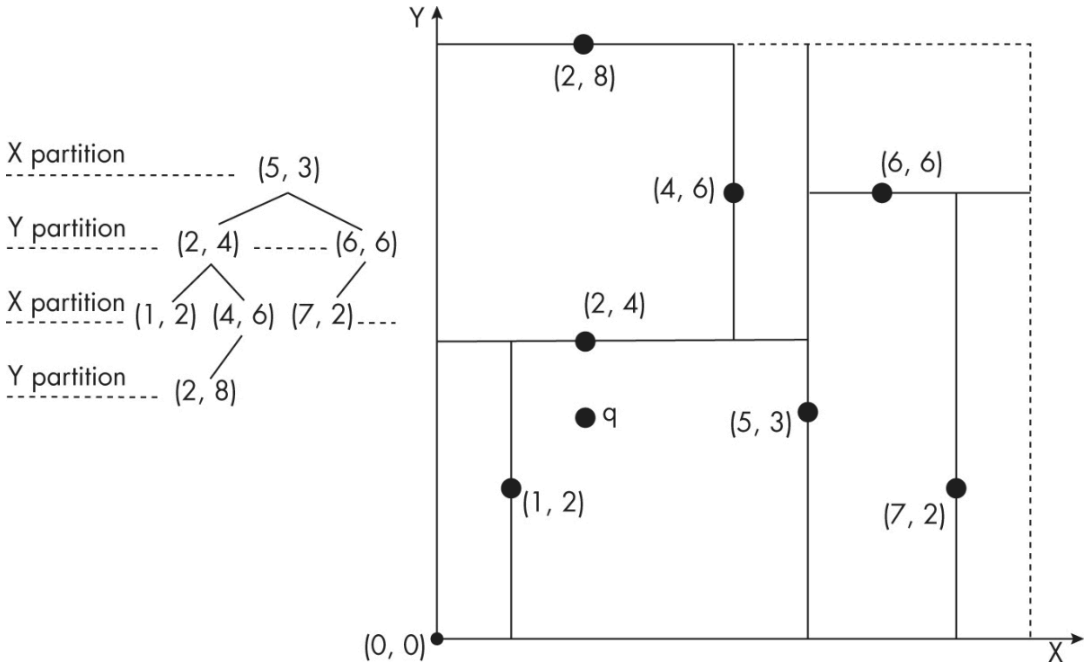

Figure 7-3: *An example of a k-d tree*

The top image of **Figure 7-3** shows the space partitioning scheme for the tree we just discussed. Starting with point (5, 3), you split the space in two along the x-dimension by drawing a vertical line through that point. Next, you use point (2, 4) to split the left half of the first partition along the y-dimension by drawing a horizontal line through the point, stopping when the line hits the vertical line. Continue in this fashion with the remaining points, and you'll get the partitioning scheme shown in the figure.

Why should you care about k-d trees? The answer is that once you arrange a dataset this way, you can search through it much more quickly. Specifically, a *nearest-neighbor search*—finding the point closest to a queried point—is much faster with a k-d tree than a linear search. For a dataset of N values, the average nearest-neighbor search of a k-d tree takes a time proportional to $\log(N)$, much less than the time proportional to N that a linear search would take.

To demonstrate, let's try to find the point from P nearest to point q, (2, 3), which is shown in **Figure 7-3**. Looking at the figure, you can see that point (2, 4) is the match. The nearest-neighbor algorithm will find

the match by traversing down the tree from (5, 3) to (2, 4). The algorithm knows, for example, that the right subtree of the root can be skipped, since *q*'s x-coordinate is less than the root node's x-coordinate. The spatial partitioning scheme thus lets you skip a larger number of comparisons than with a linear search. This is what makes the k-d tree useful for our problem.

How can you use a k-d tree in the photomosaic code? You could try to write an implementation from scratch, but there's an easier option: the `scipy` library already has a built-in k-d tree class. We'll look at how to leverage this class later in the chapter.

Requirements

For this project, you'll use `Pillow` to read in the images, access their underlying data, and create and modify the images. You'll also use `numpy` to manipulate image data and `scipy` to search the image data using a k-d tree.

The Code

You'll begin by reading in the input images that you'll draw on to create the photomosaic. Next, you'll compute the average RGB value of the images, split the target into a grid, and find the image that best matches each tile in the grid. Finally, you'll assemble the image tiles to create the actual photomosaic. To see the complete project code, skip ahead to **"The Complete Code"** on **page 129**. You can also find the code at ***https://github.com/mkvenkit/pp2e/tree/main/photomosaic***.

Reading In the Input Images

First read in the input images from a given folder. Here's how to do that:

```
def getImages(imageDir):
```

```
    """
    given a directory of images, return a list of Images
    """
❶   files = os.listdir(imageDir)
    images = []
    for file in files:
❷     filePath = os.path.abspath(os.path.join(imageDir, file))
      try:
        # explicit load so we don't run into resource crunch
❸       fp = open(filePath, "rb")
        im = Image.open(fp)
        images.append(im)
        # force loading the image data from file
❹       im.load()
        # close the file
❺       fp.close()
      except:
        # skip
        print("Invalid image: %s" % (filePath,))
```

```
    return images
```

You first use `os.listdir()` to gather the filenames in the *imageDir* directory in a list called `files` ❶. Next, you iterate through each file in the list and load it into a PIL `Image` object.

You use `os.path.abspath()` and `os.path.join()` to get the complete filename of the image ❷. This idiom is commonly used in Python to ensure that your code will work with both relative paths (for example, *\foo\bar*) and absolute paths (*c:\foo\bar*), as well as across operating systems with different directory-naming conventions (\ in Windows versus / in Linux).

To load the files into PIL `Image` objects, you could pass each filename to the `Image.open()` method, but if your photomosaic folder had hundreds or thousands of images, doing so would be highly resource intensive. Instead, you can use Python to open each image and pass the file handle `fp` into PIL using `Image.open()`. Once the image has been loaded, close the file handle and release the system resources.

You open the image file using `open()` ❸ and then pass the handle to `Image.open()` and store the resulting image, `im`, in a list called `images`. Calling `Image.load()` ❹ force-loads the image data inside `im` because `open()` is a lazy operation. It identifies the image but doesn't actually read all the image data until you try to use the image. You finish by closing the file handle to release system resources ❺.

Calculating the Average Color Value of an Image

Once you've read in the input images, you need to calculate each image's average color value. You also need to find the average color value for each section of the target image. Create a function `getAverageRGB()` to handle both tasks.

```
def getAverageRGB(image):
```

```
    """
    return the average color value as (r, g, b) for each input image
    """
    # get each tile image as a numpy array
❶   im = np.array(image)
    # get the shape of each input image
❷   w,h,d = im.shape
    # get the average RGB value
❸   return tuple(np.average(im.reshape(w*h, d), axis=0))
```

The function takes in an `Image` object—it could be one of the input images or a section of the target image—and uses `numpy` to convert it into a data array ❶. The resulting `numpy` array has the shape (`w`, `h`, `d`), where `w` is the width of the image, `h` is the height, and `d` is the depth, which, in the case of RGB images, is three units (one each for R, G, and B). You store the `shape` tuple ❷ and then compute the average RGB value by reshaping the array into a more convenient form with shape (`w*h`, `d`) so that you can compute the average using `numpy.average()` ❸. (You performed a similar operation in **Chapter 6** to get the average brightness of a grayscale image.) You return the result as a tuple.

Splitting the Target Image into a Grid

Now you need to split the target image into an *M*×*N* grid of smaller images. Let's create a function to do that:

```
def splitImage(image, size):
```

```
    """
    given the image and dimensions (rows, cols), return an m*n list of
images
    """
❶ W, H = image.size[0], image.size[1]
❷ m, n = size
❸ w, h = int(W/n), int(H/m)
    # image list
    imgs = []
    # generate a list of images
    for j in range(m):
        for i in range(n):
            # append cropped image
❹           imgs.append(image.crop((i*w, j*h, (i+1)*w, (j+1)*h)))
    return imgs
```

First you gather the dimensions of the target image ❶ and the grid size ❷. Then you calculate the dimensions of each tile in the target image using basic division ❸. Next you need to iterate through the grid dimensions and cut out and store each tile as a separate image. Calling `image.crop()` ❹ crops out a portion of the image using the upper-left and lower-right image coordinates as arguments (as discussed in **“Splitting the Target Image”** on **page 115**). You end up with a list of

images—first, all the images in the first row of the grid, from left to right; then all the images in the second row of the grid; and so on.

Finding the Best Match for a Tile

Now let's find the best match for a tile from the folder of input images. We'll look at two ways of doing this: using a linear search and using a k-d tree. For the linear search method, you create a utility function, `getBestMatchIndex()`, as follows:

```
def getBestMatchIndex(input_avg, avgs):
    """
    return index of the best image match based on average RGB value distance
    """
    # input image average
    avg = input_avg
    # get the closest RGB value to input, based on RGB distance
    index = 0
❶ min_index = 0
❷ min_dist = float("inf")
❸ for val in avgs:
❹ dist = ((val[0] - avg[0])*(val[0] - avg[0]) +
            (val[1] - avg[1])*(val[1] - avg[1]) +
```

```
            (val[2] - avg[2])*(val[2] - avg[2]))
    ❺ if dist < min_dist:
        min_dist = dist
        min_index = index
    index += 1
  return min_index
```

You're trying to search `avgs`, a list of the average RGB values of the input images, to find the one closest to `input_avg`, the average RGB value of one of the tiles in the target image. To start, you initialize the closest match index to 0 ❶ and the minimum distance to infinity ❷. Then you loop through the values in the list of averages ❸ and start computing distances ❹ using the standard formula shown in **"Matching Images"** on **page 116**. (You skip taking the square root to reduce computation time.) If the computed distance is less than the stored minimum distance `min_dist`, it's replaced with the new minimum distance ❺. This test will always pass the first time, since any distance will be less than infinity. At the end of the iteration, `min_index` is the index of the average RGB value from the `avgs` list that is closest to `input_avg`. Now you can use this index to select the matching image from the list of input images.

Now let's find the best matches using a k-d tree instead of a linear search. Here's the function:

```
def getBestMatchIndicesKDT(qavgs, kdtree):
  """
  return indices of best Image matches based on RGB value distance
```

```
    uses a k-d tree
    """
    # e.g., [array([2.]), array([9], dtype=int64)]
❶   res = list(kdtree.query(qavgs, k=1))
❷   min_indices = res[1]
    return min_indices
```

The `getBestMatchIndicesKDT()` function takes two arguments: `qavgs` is the list of average RGB values for each tile in the target image, and `kdtree` is the `scipy KDTree` object created using a list of average RGB values from the input images. (We'll be creating the `KDTree` object in **"Creating the Photomosaic"** on **page 124**.) You use the `KDTree` object's `query()` method to get the points in the tree that are closest to the ones in `qavgs` ❶. Here, the `k` parameter is the number of nearest neighbors to the queried point you want to return. You just need the closest match, so you pass in `k=1`. The return value from the `query()` method is a tuple consisting of two `numpy` arrays with the distances and indices of the matches. You need the indices, so you pick the second value from the result ❷.

Notice that the `query()` method ❶ allows you to pass in a list of query points instead of just one. This actually runs faster than querying results one by one, and it means you'll have to call the `getBestMatchIndicesKDT()` function only once, whereas you'll have to call the linear search `getBestMatch()` function many times, once for each tile in the photomosaic.

The complete program will include an option to choose which of the previous two functions to use, the linear search version or the k-d tree version. It will also have a timer to test which search method is faster.

Creating an Image Grid

You need one more utility function before moving on to photomosaic creation. The `createImageGrid()` function will create a grid of images of size $M \times N$. This image grid is the final photomosaic image, created from the list of selected input images.

```
def createImageGrid(images, dims):
    """
    given a list of images and a grid size (m, n), create a grid of images
    """
❶   m, n = dims
    # sanity check
    assert m*n == len(images)
    # get the maximum height and width of the images
    # don't assume they're all equal
❷   width = max([img.size[0] for img in images])
    height = max([img.size[1] for img in images])
    # create the target image
❸   grid_img = Image.new('RGB', (n*width, m*height))
    # paste the tile images into the image grid
    for index in range(len(images)):
```

```
    ❹ row = int(index/n)

    ❺ col = index - n*row

    ❻ grid_img.paste(images[index], (col*width, row*height))

  return grid_img
```

The function takes two parameters: a list of images (the input images you chose based on the closest RGB match to the individual tiles of the target image) and a tuple with the photomosaic's dimensions (the number of rows and columns you want it to have). You gather the dimensions of the grid ❶ and then use `assert` to see whether the number of images supplied to `createImageGrid()` matches the grid size. (The `assert` method checks assumptions in your code, especially during development and testing.) Then you compute the maximum width and height of the selected images ❷, since they may not all be the same size. You'll use these maximum dimensions to set the standard tile size for the photomosaic. If an input image won't completely fill a tile, the spaces between the tiles will show as solid black by default.

Next, you create an empty `Image` sized to fit all images in the grid ❸; you'll paste the tile images into this. Then you fill the image grid by looping through the selected images and pasting them into the appropriate spot on the grid using the `Image.paste()` method ❻. The first argument to `Image.paste()` is the `Image` object to be pasted, and the second is the top-left coordinate. Now you need to figure out in which row and column to paste an input image into the image grid. To do so, you express the image index in terms of rows and columns. The index of a tile in the image grid is given by $N \times row + col$, where N is the number of cells along the width and (*row*, *col*) is the coordinate in the grid; at ❹, you determine the row from the previous formula, and at ❺, the column.

Creating the Photomosaic

Now that you have all the required utilities, let's write the main function that creates the photomosaic. Here's the start of the function:

```
def createPhotomosaic(target_image, input_images, grid_size,
                      reuse_images, use_kdt):
    """
    creates photomosaic given target and input images
    """
    print('splitting input image...')
    # split target image
❶   target_images = splitImage(target_image, grid_size)
    print('finding image matches...')
    # for each target image, pick one from input
    output_images = []
    # for user feedback
    count = 0
❷   batch_size = int(len(target_images)/10)
    # calculate input image averages
    avgs = []
❸   for img in input_images:
```

```
        avgs.append(getAverageRGB(img))
    # compute target averages
    avgs_target = []
❹   for img in target_images:
        # target subimage average
        avgs_target.append(getAverageRGB(img))
```

The `createPhotomosaic()` function takes as input the target image, the list of input images, the size of the generated photomosaic (number of rows and columns), and flags indicating whether an image can be reused and whether to use a k-d tree to search for image matches. The function begins by calling `splitImage()` ❶ to split the target into a grid of smaller image tiles. Once the image is split, you're ready to start finding matches for each tile from the images in the input folder. Because this process can be lengthy, however, it's a good idea to provide feedback to users to let them know that the program is still working. To help with this feedback, you set `batch_size` to one-tenth the total number of tile images ❷. The choice of one-tenth is arbitrary and simply a way for the program to say "I'm still alive." Each time the program processes a tenth of the images, it will print a message indicating that it's still running.

To find image matches, you need the average RGB values. You iterate over the input images ❸ and use your `getAverageRGB()` function to compute the average RGB value for each one, storing the results in the list `avgs`. Then you do the same for each tile in the target image ❹, storing the average RGB values into the list `avgs_target`.

The function continues with an `if...else` statement to find RGB matches using either a k-d tree or a linear search. Let's look at the `if` branch first, which runs if the `use_kdt` flag was set to `True`:

```
  # use k-d tree for average match?
  if use_kdt:
    # create k-d tree
  ❶ kdtree = KDTree(avgs)
    # query k-d tree
  ❷ match_indices = getBestMatchIndicesKDT(avgs_target, kdtree)
    # process matches
  ❸ for match_index in match_indices:
    ❹ output_images.append(input_images[match_index])
```

You create a `KDTree` object using the list of average RGB values from the input images ❶ and retrieve the indices of the best matches by passing in `avgs_target` and the `KDTree` object to your `getBestMatchIndicesKDT()` helper function ❷. Then you iterate through all the matching indices ❸, find the corresponding input images, and append them to the list `output_images` ❹.

Next, let's look at the `else` branch, which performs a linear search for matches:

```
  else:
    # use linear search
  ❶ for avg in avgs_target:
      # find match index
    ❷ match_index = getBestMatchIndex(avg, avgs)
```

```
❸ output_images.append(input_images[match_index])

 # user feedback

❹ if count > 0 and batch_size > 10 and count % batch_size == 0:

    print('processed %d of %d...' %(count, len(target_images)))

 count += 1

 # remove selected image from input if flag set

❺ if not reuse_images:

    input_images.remove(match)
```

For the linear search, you start iterating through the average RGB values of the target image tiles ❶. For each tile, you search for the closest match in the list of averages for the input images using `getBestMatchIndex()` ❷. The result is returned as an index, which you use to retrieve the `Image` object and store it in the `output_images` list ❸. For every `batch_size` number of images processed ❹, you print a message to the user. If the `reuse_images` flag is set to `False` ❺, you remove the selected input image from the list so that it won't be reused in another tile. (This works best when you have a wide range of input images to choose from.)

All that remains in the `createPhotomosaic()` function is to arrange the input images into the final photomosaic:

```
 print('creating mosaic...')

 # draw mosaic to image

❶ mosaic_image = createImageGrid(output_images, grid_size)
```

```
    # return mosaic

    return mosaic_image
```

You use the `createImageGrid()` function to build the photomosaic ❶. Then you return the resulting image as `mosaic_image`.

Writing the main() Function

The `main()` function of the program takes in and parses command line arguments, loads all the images, and does some additional setup. Then it calls the `createPhotomosaic()` function and saves the resulting photomosaic. As the photomosaic is built, Python times how long the process takes, allowing you to compare the performance of the k-d tree with that of the linear search.

Adding the Command Line Options

The `main()` function supports these command line options:

```
    # parse arguments

    parser = argparse.ArgumentParser(description='Creates a photomosaic from

                                     input images')

    # add arguments

    parser.add_argument('--target-image', dest='target_image',
required=True)

    parser.add_argument('--input-folder', dest='input_folder',
required=True)

    parser.add_argument('--grid-size', nargs=2, dest='grid_size',
```

```
                required=True)

    parser.add_argument('--output-file', dest='outfile', required=False)

    parser.add_argument('--kdt', action='store_true', required=False)
```

This code contains three required command line parameters: the name of the target image, the name of the input folder of images, and the grid size. The fourth parameter is for the optional filename for the output. If the filename is omitted, the photomosaic will be written to a file named *mosaic.png*. The fifth argument is a Boolean flag that enables the k-d tree search instead of linear search for matching average RGB values.

Controlling the Size of the Photomosaic

Once all the images are loaded, one issue to address in the `main()` function is the size (in pixels) of the resulting photomosaic. If you were to blindly paste the input images together based on matching tiles in the target, you could end up with a huge photomosaic that is much bigger than the target. To avoid this, resize the input images to match the size of each tile in the grid. (This has the added benefit of speeding up the average RGB computation since you'll be using smaller images.) Here's the section of the `main()` function that handles this task:

```
    print('resizing images...')

    # for given grid size, compute the maximum width and height of
tiles

❶ dims = (int(target_image.size[0]/grid_size[1]),

        int(target_image.size[1]/grid_size[0]))

    print("max tile dims: %s" % (dims,))
```

```
    # resize

    for img in input_images:

      ❷ img.thumbnail(dims)
```

You compute the target dimensions based on the specified number of rows and columns in the grid ❶; then you use the PIL `Image.thumbnail()` method to resize the input images to fit those dimensions ❷.

Timing the Performance

When the program is run, you'll want to know how long it takes to execute. Use the Python `timeit` module for this purpose. The approach for finding execution time is outlined here:

```
import timeit

# start timing

❶ start = timeit.default_timer()

# run some code here...

-- snip --

# stop timing

❷ stop = timeit.default_timer()

print('Execution time: %f seconds' % (stop - start, ))
```

You record the start time using the `timeit` module's default timer ❶. Then, after running some code, you record the stop time ❷. Computing the difference gives you the execution time measured in seconds.

Running the Photomosaic Generator

Let's first run the program using the default linear search approach. The photomosaic will consist of a grid of 128×128 images:

```
$ python photomosaic.py --target-image test-data/cherai.jpg --input-folder
  test-data/set6/ --grid-size 128 128
```

reading input folder...

starting photomosaic creation...

resizing images...

max tile dims: (23, 15)

splitting input image...

finding image matches...

processed 1638 of 16384...

processed 3276 of 16384...

processed 4914 of 16384...

processed 6552 of 16384...

processed 8190 of 16384...

processed 9828 of 16384...

processed 11466 of 16384...

processed 13104 of 16384...

```
processed 14742 of 16384...

processed 16380 of 16384...

creating mosaic...

saved output to mosaic.png

done.

Execution time:    setup: 0.402047 seconds

❶ Execution time: creation: 2.123931 seconds

Execution time:    total: 2.525978 seconds
```

Figure 7-4(a) shows the target image, and Figure 7-4(b) shows the resulting photomosaic. You can see a close-up of the photomosaic in Figure 7-4(c). As you can see in the output, it takes about 2.1 seconds ❶ to find the best match for each of the 16,384 tiles in the photomosaic using a linear search. That's not bad, but we can do better.

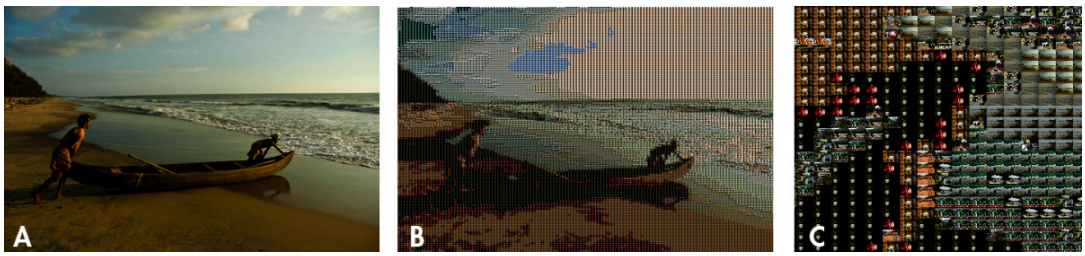

Figure 7-4: *A sample run of the photomosaic generator*

Now run the same program with the `--kdt` option, which enables the use of a k-d tree to search for image matches. Here are the results:

```
$ python photomosaic.py --target-image test-data/cherai.jpg --input-folder
```

```
  test-data/set6/ --grid-size 128 128 --kdt
```

reading input folder...

starting photomosaic creation...

resizing images...

max tile dims: (23, 15)

splitting input image...

finding image matches...

creating mosaic...

saved output to mosaic.png

done.

Execution time: setup: 0.410334 seconds

❶ Execution time: creation: 1.089237 seconds

Execution time: total: 1.499571 seconds

The photomosaic creation time has dropped from about 2.1 seconds to less than 1.1 seconds ❶ with a k-d tree. That's almost twice the speed!

Summary

In this project, you learned how to create a photomosaic, given a target image and a collection of input images. When viewed from a distance, the photomosaic looks like the original image, but up close, you can see the individual images that make up the mosaic. You also learned about an interesting data structure, the k-d tree, which signifi-

cantly sped up the process of finding the closest match for each tile in the mosaic.

Experiments!

Here are some ways to further explore photomosaics:

1. Write a program that creates a blocky version of any image, similar to **Figure 7-1**.
2. With the code in this chapter, you created the photomosaic by pasting the matched images without any gaps in between. A more artistic presentation might include a uniform gap of a few pixels around each tile image. How would you create the gap? (Hint: factor in the gaps when computing the final image dimensions and when pasting the images in `createImageGrid()`.)

The Complete Code

Here's the complete code for the project:

```
"""

photomosaic.py

Creates a photomosaic given a target image and a folder of input
images.

Author: Mahesh Venkitachalam

"""

import os, random, argparse

from PIL import Image
```

```
import numpy as np
from scipy.spatial import KDTree
import timeit
def getAverageRGBOld(image):
    """
    given PIL Image, return average value of color as (r, g, b)
    """
    # no. of pixels in image
    npixels = image.size[0]*image.size[1]
    # get colors as [(cnt1, (r1, g1, b1)), ...]
    cols = image.getcolors(npixels)
    # get [(c1*r1, c1*g1, c1*g2), ...]
    sumRGB = [(x[0]*x[1][0], x[0]*x[1][1], x[0]*x[1][2]) for x in cols]
    # calculate (sum(ci*ri)/np, sum(ci*gi)/np, sum(ci*bi)/np)
    # the zip gives us [(c1*r1, c2*r2, ...), (c1*g1, c1*g2, ...), ...]
    avg = tuple([int(sum(x)/npixels) for x in zip(*sumRGB)])
    return avg
def getAverageRGB(image):
    """
```

```
    given PIL Image, return average value of color as (r, g, b)
    """
    # get image as numpy array
    im = np.array(image)
    # get shape
    w,h,d = im.shape
    # get average
    return tuple(np.average(im.reshape(w*h, d), axis=0))
def splitImage(image, size):
    """
    given Image and dims (rows, cols) returns an m*n list of Images
    """
    W, H = image.size[0], image.size[1]
    m, n = size
    w, h = int(W/n), int(H/m)
    # image list
    imgs = []
    # generate list of images
    for j in range(m):
```

```
        for i in range(n):
            # append cropped image
            imgs.append(image.crop((i*w, j*h, (i+1)*w, (j+1)*h)))
    return imgs
def getImages(imageDir):
    """
    given a directory of images, return a list of Images
    """
    files = os.listdir(imageDir)
    images = []
    for file in files:
        filePath = os.path.abspath(os.path.join(imageDir, file))
        try:
            # explicit load so we don't run into resource crunch
            fp = open(filePath, "rb")
            im = Image.open(fp)
            images.append(im)
            # force loading image data from file
            im.load()
```

```
            # close the file
            fp.close()
        except:
            # skip
            print("Invalid image: %s" % (filePath,))
    return images
def getBestMatchIndex(input_avg, avgs):
    """
    return index of best Image match based on RGB value distance
    """
    # input image average
    avg = input_avg
    # get the closest RGB value to input, based on x/y/z distance
    index = 0
    min_index = 0
    min_dist = float("inf")
    for val in avgs:
        dist = ((val[0] - avg[0])*(val[0] - avg[0]) +
                (val[1] - avg[1])*(val[1] - avg[1]) +
```

```
                (val[2] - avg[2])*(val[2] - avg[2]))
            if dist < min_dist:
                min_dist = dist
                min_index = index
            index += 1
        return min_index
    def getBestMatchIndicesKDT(qavgs, kdtree):
        """
        return indices of best Image matches based on RGB value distance
        using a k-d tree
        """
        # e.g., [array([2.]), array([9], dtype=int64)]
        res = list(kdtree.query(qavgs, k=1))
        min_indices = res[1]
        return min_indices
    def createImageGrid(images, dims):
        """
        given a list of images and a grid size (m, n), create
        a grid of images
```

```
    """

    m, n = dims

    # sanity check

    assert m*n == len(images)

    # get max height and width of images

    # i.e., not assuming they are all equal

    width = max([img.size[0] for img in images])

    height = max([img.size[1] for img in images])

    # create output image

    grid_img = Image.new('RGB', (n*width, m*height))

    # paste images

    for index in range(len(images)):

        row = int(index/n)

        col = index - n*row

        grid_img.paste(images[index], (col*width, row*height))

    return grid_img

def createPhotomosaic(target_image, input_images, grid_size,

                      reuse_images, use_kdt):

    """
```

```
    creates photomosaic given target and input images

    """

    print('splitting input image...')

    # split target image

    target_images = splitImage(target_image, grid_size)

    print('finding image matches...')

    # for each target image, pick one from input

    output_images = []

    # for user feedback

    count = 0

    batch_size = int(len(target_images)/10)

    # calculate input image averages

    avgs = []

    for img in input_images:

        avgs.append(getAverageRGB(img))

    # compute target averages

    avgs_target = []

    for img in target_images:

        # target subimage average
```

```
        avgs_target.append(getAverageRGB(img))

    # use k-d tree for average match?
    if use_kdt:
        # create k-d tree
        kdtree = KDTree(avgs)
        # query k-d tree
        match_indices = getBestMatchIndicesKDT(avgs_target, kdtree)
        # process matches
        for match_index in match_indices:
            output_images.append(input_images[match_index])
    else:
        # use linear search
        for avg in avgs_target:
            # find match index
            match_index = getBestMatchIndex(avg, avgs)
            output_images.append(input_images[match_index])
            # user feedback
            if count > 0 and batch_size > 10 and count % batch_size == 0:
                print('processed {} of {}...'.format(count,
```

```
                                    len(target_images)))
            count += 1
            # remove selected image from input if flag set
            if not reuse_images:
                input_images.remove(match)
    print('creating mosaic...')
    # draw mosaic to image
    mosaic_image = createImageGrid(output_images, grid_size)
    # return mosaic
    return mosaic_image
# gather our code in a main() function
def main():
    # command line args are in sys.argv[1], sys.argv[2]...
    # sys.argv[0] is the script name itself and can be ignored
    # parse arguments
    parser = argparse.ArgumentParser(description='Creates a
photomosaic
                                     from input images')
    # add arguments
```

```
    parser.add_argument('--target-image', dest='target_image',
required=True)

    parser.add_argument('--input-folder', dest='input_folder',
required=True)

    parser.add_argument('--grid-size', nargs=2, dest='grid_size',

                        required=True)

    parser.add_argument('--output-file', dest='outfile', required=False)

    parser.add_argument('--kdt', action='store_true', required=False)

    args = parser.parse_args()

    # start timing

    start = timeit.default_timer()

    ###### INPUTS ######

    # target image

    target_image = Image.open(args.target_image)

    # input images

    print('reading input folder...')

    input_images = getImages(args.input_folder)

    # check if any valid input images found

    if input_images == []:
```

```
        print('No input images found in %s. Exiting.' %
(args.input_folder, ))

        exit()

    # shuffle list - to get a more varied output?

    random.shuffle(input_images)

    # size of grid

    grid_size = (int(args.grid_size[0]), int(args.grid_size[1]))

    # output

    output_filename = 'mosaic.png'

    if args.outfile:

        output_filename = args.outfile

    # reuse any image in input

    reuse_images = True

    # resize the input to fit original image size?

    resize_input = True

    # use k-d trees for matching

    use_kdt = False

    if args.kdt:

        use_kdt = True
```

```
##### END INPUTS #####

print('starting photomosaic creation...')

# if images can't be reused, ensure m*n <= num_of_images

if not reuse_images:

    if grid_size[0]*grid_size[1] > len(input_images):

        print('grid size less than number of images')

        exit()

# resizing input

if resize_input:

    print('resizing images...')

    # for given grid size, compute max dims w,h of tiles

    dims = (int(target_image.size[0]/grid_size[1]),

            int(target_image.size[1]/grid_size[0]))

    print("max tile dims: %s" % (dims,))

    # resize

    for img in input_images:

        img.thumbnail(dims)

# setup time

t1 = timeit.default_timer()
```

```
    # create photomosaic

    mosaic_image = createPhotomosaic(target_image, input_images,
grid_size,

                                     reuse_images, use_kdt)

    # write out mosaic

    mosaic_image.save(output_filename, 'PNG')

    print("saved output to %s" % (output_filename,))

    print('done.')

    # creation time

    t2 = timeit.default_timer()

    print('Execution time:    setup: %f seconds' % (t1 - start, ))

    print('Execution time: creation: %f seconds' % (t2 - t1, ))

    print('Execution time:    total: %f seconds' % (t2 - start, ))

# standard boilerplate to call the main() function to begin

# the program.

if __name__ == '__main__':

    main()
```
